

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_6237b110-d58\agent-aaf6e405553eb5094.jsonl`  
- SHA-256: `51e171e83436a08601aed50a65c5e7ee54eba826789813a558adff3107a20392`  
- Source modified: `2026-06-06T17:28:43+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_6237b110-d58`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T17:27:51.905Z)

Decompose this research question into complementary search angles.

Question

Research alternative methods to automatically detect rally start/end segments (temporal segmentation) in racket-sport video ? especially badminton ? for a SELF-HOSTED, LOCAL-FIRST pipeline, and assess whether a cloud-VLM teacher ? local-model distillation is the best approach or whether other methods are competitive or complementary. Keep it focused ("mini" research): prioritize actionable, commercially-licensable, locally-runnable approaches over exotic SOTA needing huge compute or restrictive licenses.

OUR CONTEXT / CONSTRAINTS (ground every recommendation in these):

- Self-hosted on a single consumer GPU (RTX 2070 Super), Windows + WSL2. We want LOCAL-ONLY inference at runtime ? privacy lever: user video never leaves the device. It's a commercial product, so PERMISSIVE / commercial-friendly licenses are REQUIRED (avoid AGPL and non-commercial-only weights/datasets).
- Current substrate: WASB (MIT) shuttle-trajectory detector ? per-frame (x, y, visibility). We derive rally windows via velocity/merge-gap/min-duration thresholds plus a net-crossing gate. We ALSO have a permissive person detector available (torchvision Faster R-CNN + supervision ByteTrack) and the video's AUDIO track.
- Current best approach: use paid Gemini (cloud VLM, full-context) OFFLINE as a TEACHER to label rallies, then DISTILL into a local model (logistic regression on shuttle-trajectory features). Findings so far: threshold-tuning ceilings around F1 ? 0.44 and overfits with few videos; the learned model is data-starved at n=2 labeled videos; WASB candidate RECALL is a real bottleneck (its windows sometimes don't align with true rallies at IoU 0.5).
- Inputs we can exploit locally: video frames, the shuttle trajectory, player bounding boxes / pose (if we add a permissive pose model), and the AUDIO track (shuttle "thwack" hits, crowd noise, umpire/score calls).

SPECIFICALLY COVER:

1. Method families for rally/point segmentation in racket sports (badminton, tennis, table tennis, pickleball): audio-based shot/hit detection; temporal action localization (TAL) deep models; pose/player-stance + court-homography state machines (e.g., "serve-ready stance" gating); classical change-point detection / HMM on the shuttle trajectory; scoreboard / score-overlay OCR for point boundaries; self-supervised / weakly-supervised approaches; and VLM teacher?student distillation.
2. For EACH family: how it works, the labels/data it needs, expected accuracy, local-inference COMPUTE cost, and LICENSE / commercial viability of any models, datasets, or weights involved.
3. Badminton rally-segmentation datasets (e.g., ShuttleSet, TrackNet/TrackNetV2 data) and their licenses / commercial usability.
4. A RANKED recommendation for our local-first, permissive, single-GPU setup: is Gemini-teacher ? local-distillation the best, or are some alternatives competitive or complementary ? especially AUDIO-based hit detection and pose/court-geometry state machines, which we can run fully locally?
5. Concrete, cheap next experiments we could run with what we already have (shuttle trajectory +

person detector + audio).

Task

Generate 5 distinct web search queries that together cover the question from different angles.

Pick angles that suit the question's domain. Examples:

- broad/primary · academic/technical · recent news · contrarian/skeptical · practitioner/implementation
- For medical: anatomy · common causes · serious differentials · authoritative refs · red flags
- For tech: state-of-art · benchmarks · limitations · industry adoption · cost/tradeoffs

Make queries specific enough to surface high-signal results. Avoid redundancy.

Return: the question (verbatim or lightly normalized), a 1-2 sentence decomposition strategy, and the angles.

Structured output only.

2. assistant (2026-06-06T17:28:02.610Z)

I'll decompose this research question into complementary search angles. Let me first check the project context to ground the angles in the actual work.

3. user (2026-06-06T17:28:04.431Z)

```
1      ---
2      name: current-state
3      description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
4      only on request). What just landed, what's next, open decisions."
5      metadata:
6      node_type: memory
7      type: project
8      originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
9      ---
10     # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12     **Checkpoint:** 2026-06-06 (late) · Gemini teacher?student pipeline built. DECISION:
13     PAUSE precision work, WAIT for next week's videos. Open PRs #27-#30. See resumption runbook ?.
14
15     ## DISTILLED MODEL BUILT (2026-06-06) ? user wants this approach; honest result = needs
16     more data
17     - PRs #27-#30 MERGED to master (#30 stranded calibrate_local on gate branch ? RECOVERED
18     in PR #31).
19     - `backend/eval/distill_local.py` (PR #31, OPEN): learned model ? WASB recall-oriented
20     candidates (0.005,1.0,0.5)
21     ? features [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]
22     (resolution/fps-INVARIANT)
23     ? classifier.LogisticRegression trained on Gemini labels (training.label_candidates
24     IoU). Honest LOVO + saves
25     output/local_rally_model.json. Reuses
26     classifier/training/rally_gate/metrics/gemini_refine.
27     - **HONEST RESULT (2 videos): LEARNED UNDERPERFORMS baseline** ? mean held-out F1 0.359
28     (learned) vs 0.438
29     (threshold baseline). Causes: (1) n=2 too few ? no transferable model; (2) WASB
30     proposal recall CEILING 0.43
31     on TestLargeVideo (its windows don't align with Gemini rallies; Adarsh ceiling 0.81).
32     Framework READY; needs
33     more teacher videos + better WASB recall to beat baseline. Final weights:
34     mean_velocity + duration dominate,
35     net_crossings barely used (unreliable at n=2).
36     - ? **VeryLargeTestVideo (4K, 46080 frames, 12m49s, 11.5GB) = held-out FINAL test for
37     USER MANUAL EVAL.**
38     Extraction running (task bzt8hc4ks, frames?1920). Then WASB ? baseline + learned rally
39     lists ? user manually
40     evals (manual = GT; no Gemini needed on it). User's manual eval becomes a 3rd-video
```

```

human GT ? strengthens model.
28     - PR #31 (feat/distill-local) OPEN ? merge to get distill_local + calibrate_local onto
master.
29
30     ## ?? NEXT-WEEK RESUMPTION RUNBOOK (more teacher videos ? re-run calibrate_local +
distill_local; learned beats baseline once n grows)
31     **Goal:** more teacher-labeled videos ? stable leave-one-video-out ? then build the
LEARNED distilled model
32     (classifier.py on Gemini labels) ? the real local-precision lever (threshold-tuning
ceiling'd at ~0.44 P/F1, overfits on 2 videos).
33     **FIRST: merge the open PR stack** so master has the tools: #27 (code-map), #28
(gate-A/rally_gate), #29
34     (gemini-refine + provider hardening + extract_video_chunk scale_width), #30
(calibrate_local, stacked on #28 ? retarget to master after #28).
35     **Per NEW video (repeat for each):**
36     1. Probe: `ffprobe ... <video>` ? note fps, width. (TestLargeVideo was 5.3K ? extract
frames downscaled to 1920.)
37     2. Extract frames: Windows `ffmpeg -i <video> [-vf scale=1920:-2] -q:v 2
<Temp>/<name>_frames/%08d.jpg`.
38     3. WASB trajectory (WSL, checkpointed): re-stage wasb_infer.py to `~/models/WASB-
SBDT/src`, then
39     `conda activate wasb && cd ~/models/WASB-SBDT/src && python wasb_infer.py
--frames_dir /mnt/c/.../<name>_frames
40     --weights ../pretrained_weights/wasb_badminton_best.pth.tar --out
~/clips/<name>_traj.csv --batch-size 8 --num-workers 4 --flush-every 500`
41     then copy CSV to a Windows path.
42     4. Gemini teacher labels (paid key in env GEMINI_API_KEY; NEVER store key): `python -m
backend.eval.gemini_refine
43     --video <video> --full-video --out output/<name>_gemini_fullvideo.csv` (model gemini-
flash-latest; clips auto-downscaled <500MB).
44     5. Add `{name,trajectory,fps,frame_width,labels}` to the calibration manifest (see
output/local_calib_manifest.json).
45     **Then re-run calibration + build the learned model:**
46     6. `python -m backend.eval.calibrate_local --manifest <manifest> --metric f1` (and
--metric precision) ? honest LOVO across N videos.
47     7. **NEW (the real lever):** train `backend/eval/classifier.py` (LogisticRegression) on
pooled Gemini labels: label each WASB
48     candidate rally-vs-not by IoU-match to Gemini labels (training.py::label_candidates),
features = candidate stats
49     (peak/mean velocity, active_frame_count) + net-crossings (rally_gate). Score held-out
(LOVO). Compare to threshold-calib.
50     **Artifacts that persist (don't redo):** trajectories
`~/clips/{adarsh,testlarge,sample_long}_traj.csv` (+ Windows Temp copies);
51     Gemini labels output/{testlarge,adarsh}_gemini_fullvideo.csv; manifest
output/local_calib_manifest.json.
52     **Deletable (big, regenerable from trajectories):** Temp/{adarsh_frames(7.9GB),
testlarge_frames, sample_long_frames(4.1GB)}.
53
54     ## OLD checkpoint context below ? (this session's journey)
55
56     ## DEMO RUN ? 2nd video "Adarsh and Vasanth.MP4" (2026-06-06, stress + checkpoint test)
57     - Video: `C:\Users\avidu\Projects\Annotation Setup\Collect\Adarsh and Vasanth.MP4`,
1920x1080, 59.94fps,
58     40,536 frames, 11m16s, 5.5GB. Stress-tested infra (1.9x Sample Long).
59     - Frames: ffmpeg -> `C:\?Temp\adarsh_frames` (40,536 JPGs, 7.9GB, deletable). Inference
40,534 windows
60     in 1272s @ 31.9 win/s; 13,224/40,536 visible (32.6%). Checkpoint cache kept pace (no
crash, so resume
61     not exercised, but cache fully built). Trajectory: WSL `~/clips/adarsh_traj.csv` +
`C:\?Temp\adarsh_traj.csv`.
62     - Tuned export (cfg 0.05/1.0/1.0, no golden labels for this video): **65 rally segments,
251.4s play** ->
63     `output/adarsh_tuned_segments.csv`. User eyeballing; baseline NOT yet generated (held
until user anchors).
64     NOTE tuned cfg was fit on Sample Long (different video) -> transfer is the open

```

question.

```

65
66     ## BASELINE vs TUNED on Adarsh (2026-06-06) ? tuned config does NOT transfer + over-fire
pattern
67     - Baseline (0.02/2.0/2.0): 45 segs, 303.8s, mean 6.8s. Tuned (0.05/1.0/1.0): 65 segs,
251.4s, mean 3.9s.
68     - Tuned = +44% segments but -17% play: 30 tuned-only segs (almost all 1-2s = false
fires); 10 baseline-only
69     segs that tuned LOST are the LONG rallies (17.7/10.9/9.6/9.2/8.9/7.6s) ? merge_gap=1.0
fragments them.
70     - **Honest verdict: no "% improvement" (no GT for this video); tuned REGRESSES here.
Baseline is the better
71     operating point on Adarsh.** Concrete proof the Sample-Long-tuned cfg overfits -> need
leave_one_video_out
72     (2nd labelled video). `output/adarsh_baseline_segments.csv` written.
73     - **OVER-FIRE PATTERN (user-documented, data-confirmed):** loser tossing/flicking
shuttle back for service =
74     single short trajectory -> WASB detects as rally start. Real rally = shuttle crosses
net MULTIPLE times.
75     - **FIX SPECTRUM (design, grounded in current substrate):** (A) shuttle-only: gate
candidates on net/midline
76     crossings >=K (?shot count) / direction-reversals ? cheap, no new model, kills single-
toss fires. (B) user's
77     START-STANCE STATE MACHINE: IDLE->await-stance(players settled+diagonal in service
zones)->ARMED->shuttle->
78     RALLY->end->IDLE; reuse the existing person detector (yolo_hybrid torchvision
FRCNN+ByteTrack) as a VERIFIER
79     run only ~2s around each candidate start (not full-video, which was ~50min/video). (C)
pose-based serve
80     detection ? richest, new model + license review, likely overkill. Rec: A now, B robust
target; B needs research.
81     - ? **GATE A SHIPPED ? PR #28** (`feat/rally-start-gate`):
`backend/detectors/rally_gate.py` (pure,
82     camera-agnostic, no new model) ? estimate play axis (larger shuttle spread) + net
(median coord),
83     count (coord-net) sign-changes per window, keep crossings>=K. Wired into export via
`--min-crossings`
84     (0=off, 2 rec). 6 tests, suite 193 green. **Validated on Adarsh (side camera auto-
detected, net x?716):
85     real rallies 5-13 crossings, junk 0-1; K>=2 kills 18 of ~19 short tuned false fires,
keeps all long
86     rallies.** `output/adarsh_baseline_gated.csv` = BASELINE+gate(K=2) = 38 segs (45->38)
= the "stronger
87     baseline now". 1 yellow flag to eyeball: 4:39.5-4:44.7 (5.2s,1 crossing) removed ?
real rally or one-sided?
88     - **STRATEGIC (user, 2026-06-06): paid-Gemini as a cost-controlled QUALITY fallback.**
User OK slicing
89     videos <1GB + paid Gemini to build a stronger baseline NOW + as a product "calculated-
bit-expensive,
90     quality-significantly-improving" fallback. KEY: the hybrid Phase-3 ALREADY does this
(WASB candidates ->
91     Gemini verifies each PADDED CANDIDATE CLIP, not whole video -> cost bounded by
#candidates*cliplen, not
92     duration); it died only on FREE-tier quota. So paid key re-enables it. Recommended
stack: WASB propose ->
93     gate A (free, cuts junk -> fewer Gemini calls) -> Gemini verify survivors (paid,
precision+boundaries).
94     ? PRIVACY TRADE-OFF: sending video to Gemini breaks the "vendors/AI never see user
uploads" privacy lever
95     ([[monetization-plan]]) ? fine for OWNED/test/golden video; for end users make it an
explicit opt-in
96     premium tier. More annotated videos arrive ~next week (-> leave_one_video_out for
honest cross-video).
97
98     ## GEMINI REFINE (2026-06-06) ? WASB candidates -> Gemini verify/refine (in progress)

```

```

99      - `backend/eval/gemini_refine.py` (NEW, uncommitted on master working tree; + provider
retry edits in
100      `backend/providers/gemini.py`). Reuses GeminiProvider + badminton prompt +
extract_video_chunk +
101      make_predict_fn + write_segments_csv (no reinvent). Drives Gemini over SHORT WASB
candidate clips
102      (~baseline 45, padded 3s, reencoded ? tiny uploads ~450MB total, NOT GB chunks).
Gemini returns the
103      true rally per clip or [] (rejects between-point feeds it can see). **Envelope per
candidate** (1 WASB
104      candidate ? 1 rally) so Gemini fragments don't shatter a long rally below min_dur.
User has a PAID key
105      (env GEMINI_API_KEY only ? NEVER store the key in memory/disk/commits).
106      - **MODEL GOTCHA (verified 2026-06-06):** `gemini-2.5-flash` works but hits frequent
**503 "high demand"**
107      spikes; `gemini-2.0-flash` is **404 (deprecated for generateContent)** despite
appearing in models.list.
108      ? use **`gemini-flash-latest`** (clean, no 503, current stable flash). Account also
has gemini-3.x /
109      gemini-3.5-flash (newer than my training). Added upload+generate retries (3x backoff)
to the provider;
110      per-call client per worker (genai client not thread-safe ? socket errors when shared).
111      - Smoke (6 candS) clean on flash-latest: tightens boundaries (e.g. WASB 21.3-27.5 ?
Gemini 23.8-25.8),
112      keeps long rally whole. Full 45-candidate run launched (task bkap9jri2) ->
output/adarsh_gemini_refined.csv.
113      - NEXT after run: compare Gemini-refined vs WASB baseline/baseline+gate (Gemini as
SILVER-GT to finally
114      quantify Adarsh); commit gemini_refine + provider retries as a PR. Recall ceiling =
WASB candidates
115      (Gemini only sees proposed windows; rallies WASB missed entirely won't appear).
116
117      ## PRECISION FINDING (2026-06-06): full-context Gemini > candidate-refine (envelope
over-merges)
118      - Added `full_video_rallies()` to gemini_refine + `--full-video` (whole clip, downscaled
1280, chunked
119      <=120s to stay under 500MB) + clip downscaling in extract_video_chunk (scale_width).
Reuses provider/prompt.
120      - **TestLargeVideo (5.3K, 100s) head-to-head:** candidate-refine = 5 rallies/49s;
**full-context = 7 rallies/
121      34.5s**. Refine collapsed the 0:23-0:43 region into ONE 22s blob; full-context split
it into 3 real rallies
122      (23.5-26.5, 31.5-33.5, 39-42.5) with dead time between. Root cause: WASB merge_gap=2.0
bundled 3 rallies
123      into 1 candidate, then my **envelope-per-candidate** collapsed Gemini's fragments back
into 1 ? swept ~13s
124      dead time into a "rally" = PRECISION LOSS. Full-context is a strict improvement (same
rallies, tighter, +
125      correct split). Files: output/testlarge_gemini_rallies.csv (refine),
output/testlarge_gemini_fullvideo.csv (full).
126      - ? **USER CONFIRMED (2026-06-06): full-context is correct** ? 0:23-0:43 IS three
rallies (24-26, 31-34,
127      39-42), not one blob; later rallies also right. Full-context is the precision-improved
default.
128      - ? **FIXED + SHIPPED ? PR #29** (`feat/gemini-refine`): dropped the over-merging
envelope; tiny
129      invisibility splits healed by gap-tolerant merge (<=1s), real between-rally gaps kept
separate. Provider
130      hardening (upload+503 retries, MAX_UPLOAD_MB=500 guard), extract_video_chunk
scale_width downscaling,
131      default model gemini-flash-latest, per-worker client. 7 tests, suite 192 green. Modes:
refine (WASB
132      candidates) + `--full-video` (whole clip, chunked<=120s, downscaled).
133      - ? Adarsh full-context RUNNING (task b9fs6aylf) -> output/adarsh_gemini_fullvideo.csv
(~7 chunks).

```

```

134 - Ensemble voting (N Gemini judgments/rally, majority+median boundaries) = next
precision lever if needed.
135 - ? Gemini sees user video = breaks privacy lever ([[monetization-plan]]); OK for
owned/test video, opt-in
136 premium tier for end users.
137
138 ## ? TEACHER?STUDENT DISTILLATION (user's framing, 2026-06-06): Gemini=offline teacher,
LOCAL-ONLY WASB+gate=student
139 - GOAL: use Gemini (cloud, offline, one-time) as silver-GT to CALIBRATE the local-only
knobs so PRODUCTION
140 runs 100% local (no Gemini at runtime ? privacy lever intact, ~0 marginal cost).
Gemini quality distilled
141 into thresholds.
142 - BUILT: `backend/eval/calibrate_local.py` (+test, 2 pass). predict_fn =
trajectory_to_action_windows
143 (velocity/merge/min_dur) + rally_gate(min_crossings); grid 5x3x3x4=180; scores vs
Gemini silver labels;
144 `leave_one_video_out` across videos = honest cross-video precision.
BASELINE_LOCAL=(0.02,2.0,2.0,0).
145 **DEPENDS ON PR #28 (rally_gate)** ? built on branch `feat/local-calibration` (stacked
on feat/rally-start-gate).
146 - INPUTS: WASB trajectories (testlarge_traj.csv, adarsh_traj.csv on disk) + Gemini full-
context label CSVs
147 (output/testlarge_gemini_fullvideo.csv done=7 rallies;
output/adarsh_gemini_fullvideo.csv RUNNING b9fs6aylf).
148 - NEXT: when Adarsh labels land ? manifest of 2 videos ? run calibrate_local ? tuned
local cfg + honest LOVO
149 precision (baseline-untuned vs calibrated held-out). Commit/PR (note #28 dependency).
More videos next week
150 ? stronger LOVO.
151 - BRANCH STACK now: #27 code-map, #28 gate, #29 gemini-refine, feat/local-calibration
(on #28). Suggest user
152 merges #28+#29 so local-calibration rebases onto clean master.
153
154 ## ? RAN local calibration vs Gemini silver-GT (2 videos: TestLargeVideo=7 rallies,
Adarsh=36) ? HONEST result
155 - **Held-out (LOVO) ? the trustworthy number:**
156 . F1-optimized: baseline 0.438 ? calibrated **0.483** ±0.17, overfit gap +0.035 (small
?) = modest real lift.
157 . Precision-optimized: baseline 0.444 ? calibrated **0.320** ? ±0.10, gap **+0.245
(severe overfit)** = does NOT transfer.
158 - Per-video fit-on-self (optimistic): Adarsh P 0.489?0.731 / F1 0.543?0.683;
TestLargeVideo barely moves (F1 0.33).
159 - **CONCLUSION (honest, don't oversell):** local-only pipeline agrees with Gemini only
~0.44 P/F1; threshold-
160 tuning hits a ceiling fast and OVERFITS for precision with only 2 dissimilar videos
(5.3K phone vs Adarsh);
161 they want opposite configs. TestLargeVideo also RECALL-capped (WASB ~5 windows vs 7
Gemini rallies).
162 - **REAL LEVERS for local precision (next):** (1) MORE teacher-labeled videos (next
week) ? stable LOVO;
163 (2) LEARNED distilled model ? train existing `classifier.py` on Gemini labels using
candidate features
164 (peak/mean velocity, active_frame_count, net-crossings) instead of 4 thresholds; (3)
raise WASB recall.
165 Recommend learned-distillation over more threshold-tuning.
166
167 ## CODE_MAP artifact ? PR #27 (`docs/code-map`, OPEN) ? see [[code-map-artifact]]
168 `docs/CODE_MAP.md` = dense cold-ramp navigation map
(pipeline/subsystems/contracts/gotchas/ramp-paths/
169 test-map). Built by a map->synthesize->adversarial-verify workflow; verify caught real
errors (validator
170 order, nonexistent config.json indexing.wasb, extract_yolo_features gotcha). Regenerate
via the
171 `build-code-map` workflow. Read it FIRST for code ramp-up.

```

```

172
173     ## ? NEXT-SESSION pickup
174     - **User to eyeball** the tuned segments (`output/wasb_tuned_segments_vs_golden.csv`;
regenerate via
175     `python -m backend.eval.export_wasb_segments --trajectory
C:\?\Temp\sample_long_traj.csv --labels
176     "Annotation Setup\golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
--baseline`; add
177     `--video <mp4> --slice` for clips). Verdict drives whether to tune merge_gap
(fragmentation) /
178     min_window_duration (short rallies).
179     - **Real precision verdict** still needs a **2nd labelled video** ?
`leave_one_video_out`.
180     - Optional housekeeping: stale MERGED remote branches `feat/rally-slicer` (#20) +
`feat/wasb-substrate`
181     (#21) still on origin (never deleted; harmless, all on master). NOT deleted this
session given the
182     #21-branch-delete-lost-calibrate_wasb history ? ask before pruning. KEEP
183     `feat/khelacharya-shot-intelligence` (PR #11 closed, intentionally retained).
184
185     ## ? (1) EXPORT TUNED SEGMENTS DONE ? PR #26 MERGED to master (`feat/wasb-export-
segments`)
186     `backend/eval/export_wasb_segments.py` (+5 tests, suite 187 green). Reuses
make_predict_fn +
187     metrics.match_intervals/score + rally_slicer (exported CSV is in the golden/slicer
schema ? feeds
188     straight into clip cutting via `--slice --video`). Ran on REAL trajectory + golden
labels:
189     - **34 tuned segments** (cfg 0.05/1.0/1.0); vs 25 golden: TP 22, FN 3, FP 12 ? precision
0.647,
190     recall 0.880, **F1 0.746**, meanIoU 0.753 (reproduces calibration EXACTLY). Mean
boundary err:
191     start 0.64s / end 0.91s. Artifacts in `output/` (gitignored):
`wasb_tuned_segments.csv`,
192     `_baseline.csv`, `_vs_golden.csv`.
193     - **EYEBALL FINDINGS (honest):** recall strong, boundaries mostly <1s, but precision
0.647 = tuned
194     config OVER-segments. The 3 "misses" + 12 "FPs" are partly artifacts not pure errors:
195     . golden 3 (44.5-51.1s) FRAGMENTED into 2 sub-segments (44.8-46.9 + 48.4-50.8) ?
neither hits
196     IoU 0.5 ? 1 miss + 2 FPs from ONE rally (a merge_gap problem).
197     . golden 8 (118.1-119.6, 1.5s) detected as 118.1-123.0 but boundary too long ? 1 miss
+ 1 FP.
198     . golden 7 (95.6-98.0, 2.4s) genuinely dropped (short rally).
199     . boundary overruns: golden 1 start +6.16s (missed first 6s of a long rally), g21 end
+4.76s,
200     g23 end +3.05s.
201     . ~9 genuine extra segments to eyeball: 3.35-7.96, 36.9-39.4, 53.9-58.4, 60.5-62.9,
79.5-81.2,
202     107.2-111.2, 144.5-146.6, 188.2-190.3, 339.1-343.3 (warmup / between-point motion /
real
203     unlabeled rallies?).
204     . LEVERS if user wants better precision: raise merge_gap (fix fragmentation), revisit
205     min_window_duration for short rallies; real precision verdict needs a 2nd labelled
video
206     (`leave_one_video_out`).
207
208     ## ? PR status: #25 (checkpointing) + #26 (export) BOTH MERGED to master. No open
indexer PRs.
209
210     ## NEW session 2026-06-06 (late) ? user asked for tasks in order: (4) repo cleanup ? (2)
WASB checkpointing ? (1) export tuned segments
211     - ? ** (4) repo cleanup DONE:** collector local checkout was stranded on the merged
`feat/validate-delivery`
212     branch ? synced to `master` (ff to d68c5da = PR #9 merge) + deleted the merged branch.

```

```

Both repos now
213     genuinely clean on master. Untracked local `data/roora_*.` artifacts left in place (not
committed).
214     - ? **PR #11 (KhelAcharya shot-intel) CLOSED, branch retained.** Correction: there is NO
separate
215     khelacharya GitHub repo ? that folder is a 2nd clone of THIS repo. Separation is
branch-level; #11
216     kept shot-work off master. Closed to declutter; `feat/khelacharya-shot-intelligence`
branch persists,
217     reopenable. See [[sibling-internship-repo]] (corrected).
218     - ? **PR #25 (WASB-pass checkpointing DONE ? PR #25 MERGED to master** (`feat/wasb-
checkpointing`)).
219     `wasb_infer.py` refactored: per-window detector cache `out_wasbcache/det_raw.jsonl`
(fsyntax every
220     --flush-every=1000 windows) + atomic `manifest.json`; resume replays cache + skips
DataLoader to first
221     un-cached window; tracker re-run over full ordered cache (cheap, deterministic);
atomic tmp+replace
222     writes; trailing-corruption truncation; auto-invalidate on input change; idempotent
extract_frames.
223     11 new unit tests (`tests/test_wasb_cache.py`), suite **182 green**. WSL-validated:
tracker-only re-run
224     byte-identical; incremental resume matches full run except 3/300 rows 7e-5 px (cuDNN
float noise from
225     re-batching, NOT a bug); --limit change correctly invalidates. Runner gets resume for
free (cache
226     derives from --out, not %TEMP%). **GOTCHA:** must re-stage `wasb_infer.py` to
`~/models/WASB-SBDT/src/`
227     in WSL when running directly (runner does this automatically).
228     - ? **NEXT: (1) export tuned segments** for user eyeball ("are rallies better?"). Use
tuned cfg
229     (velocity 0.05, merge_gap 1.0, min_dur 1.0) on the cached trajectory ? cut rally
clips.
230     Artifacts confirmed intact: `~/clips/sample_long_traj.csv` (21,284 frames) + Windows
Temp copy;
231     21,284 frames in `Temp/sample_long_frames`; golden labels at `Annotation
Setup\golden_labels_badminton.csv`.
232
233     ## Merged to master (both repos clean, branches deleted)
234     - collector #6 (CVAT?canonical adapter), #7 (forced/unforced vocab), indexer #19 (docs).
235     - Local master pulled + clean in both repos.
236
237     ## WASB inference API (reverse-engineered 2026-06-06 ? for the wrapper)
238     Repo `~/models/WASB-SBDT` (Hydra configs). Usage: `cd src && python3 main.py --config-
name=eval
239     dataset=badminton model=wasb
detector.model_path=../pretrained_weights/wasb_badminton_best.pth.tar`.
240     Inference core (`src/runners/eval.py::inference_video`): build detector + tracker +
dataloader ?
241     loop batches `(imgs, hms, trans, ...)` ? `detector.run_tensor(imgs, trans)` ? per-frame
preds ?
242     `tracker.refresh()` then `tracker.update(preds)` per frame ? `result_dict[img_path] =
{x,y,visi,score}`.
243     That dict IS the trajectory we want.
244     - `TracknetV2Detector(cfg)` (`src/detectors/detector.py`): builds model via
`build_model(cfg)`,
245     loads `cfg.detector.model_path` (`checkpoint['model_state_dict']`), DataParallel,
eval. Needs
246     CUDA (asserts GPU). `frames_in`=cfg.model.frames_in, `input_wh`=(model.inp_width,
inp_height).
247     `run_tensor(imgs, affine_mats)` ? model ? postprocessor ? per-frame (x,y,visi,score).
248     Preprocessing helpers: `dataloaders.build_img_transforms/read_image/get_transform`,
249     `utils.image.get_affine_transform/affine_transform`.
250     - eval.yaml defaults: model=wasb, detector=tracknetv2, tracker=online,
transform=default.

```

```

251 - **Wrapper plan:** extract video frames ? build `frames_in` sliding windows + affine
`trans`
252 (mirror dataset `__getitem__`) ? run detector+tracker ? emit CSV
253 **`Frame,Visibility,X,Y`**
254 (EXACTLY what existing `tracknet_runner.parse_trajectory_csv` +
`trajectory_to_action_windows`
255 already consume ? big reuse). Use weights `wasb_badminton_best.pth.tar` (NOT
monotrack=noncommercial).
256 - **Build steps:** (1) `backend/detectors/wasb_infer.py` ? **DONE + VALIDATED** (branch
257 `feat/wasb-substrate`, pushed, not yet PR'd). Runs in WSL `wasb` env; reuses WASB
ImageDataset
258 transform + detector + online tracker; emits Frame,Visibility,X,Y. Verified on
rally-18 clip
259 (121 frames ? 107 visible, X439-1036/Y64-586, smooth = real shuttle track).
260 **GOTCHAS:** must run from `~/models/WASB-SBDT/src` in `wasb` env; Hydra overrides
need
261 `runner.gpus=[0]` (single-GPU box) + `runner.device=cuda`; `__getitem__` lives in
262 `dataloaders/dataset_loader.py::ImageDataset` (samples =
263 `{ 'frames': [paths], 'annos': [{frame_path,
264 center:Center(is_visible,x,y)] }`); read frames via /mnt (Google-Drive G: extraction
done on
265 Windows ffmpeg). To run: `conda activate wasb && cd ~/models/WASB-SBDT/src && python
wasb_infer.py
266 --frames_dir <dir> --weights ../pretrained_weights/wasb_badminton_best.pth.tar --out
<csv>`.
267 (2) `wasb_runner.py` ? DONE (Win adapter: stage video+script to WSL ? run `wasb` env ?
CSV;
268 REUSES `TrackNetRunner.parse_trajectory_csv`/`trajectory_to_action_windows` as
staticmethods ?
269 those are class staticmethods, NOT module funcs; only
`to_wsl_mnt_path`/`TrajectoryPoint` are
270 module-level). (3) `wasb_hybrid.py` ? DONE (mirrors tracknet_hybrid; registered
"wasb_hybrid";
271 reads thresholds from idx_cfg.wasb). All on branch `feat/wasb-substrate` (pushed, NOT
PR'd).
272 Suite 159 green; registry lists wasb_hybrid. **NOT yet end-to-end run**: full-video
WSL pass +
273 AI handoff untested (wasb_infer core IS validated on real frames). config.json needs
an
274 `indexing.wasb` section (defaults exist in WasbConfig). (4) per-video calibration
**Workflow**
275 = DEFERRED (user will call it). Calibration only needs Phase-2 windows (no AI
handoff/API).
276 Preprocessing (RESOLVED): `configs/model/wasb.yaml` ? name=hrnet, frames_in=3,
frames_out=3,
277 inp 512x288 (WxH). `dataloaders/__init__.py` build_img_transforms: T.ToTensor +
Normalize
278 mean[0.485,0.456,0.406]/std[0.229,0.224,0.225] (ImageNet). Resize via affine
(`utils.image.
279 get_affine_transform` ? input_wh; the affine matrix is the `trans`/`affine_mats`
passed to
280 run_tensor and used to map heatmap coords back to original frame). So wrapper: per
sliding window
281 of 3 consecutive frames ? affine-resize to 512x288 ? ToTensor+Normalize ? stack ?
detector.run_tensor
282 ? tracker ? (x,y,visi,score) per frame ? CSV Frame,Visibility,X,Y. Weight variant:
wasb_badminton_best.
283
284 ## GOAL CORRECTION (user clarified 2026-06-06 ? important)
285 User wants to "make the WASB pretrained WEIGHTS more robust with the 1 annotated video."
Honest reality:
286 - Our annotation = **temporal rally [start,end]** labels, NOT per-frame shuttle (x,y).
WASB is a shuttle
287 DETECTOR (needs x,y heatmap supervision). **So the rally labels CANNOT fine-tune
WASB's detector weights.**

```

```

286 - What the 1 video CAN quantifiably improve = the **rally-SEGMENTATION layer on top of
WASB**: the 3
287 windowing thresholds (velocity_thresh/merge_gap/min_window_duration) + WASB variant
choice. Measure
288 IoU/F1 on HELD-OUT golden rallies (calibrate on K, test on rest). That's the
"quantifiable improvement".
289 - To genuinely fine-tune WASB DETECTOR weights ? need per-frame shuttle (x,y) labels (a
different
290 annotation, or pseudo-labels) ? separate track, not this annotation.
291 - Also: neural weights are found by backprop, not by agents "trying weights +
coordinating". Agents are
292 the right tool for the CONFIG search (variant x 3 thresholds x strategy), not weight
space.
293
294 ## Tuning plan = a Workflow (per user's ultracode ask), runs AFTER the wrapper exists
295 Once WASB trajectories exist: run a parallel **calibration Workflow** ? agents
independently search
296 {WASB variant x velocity_thresh x merge_gap x min_window_duration x search-strategy},
each scored on
297 held-out golden rally IoU, coordinate ? pick max-precision config. (Can't run before a
trajectory exists.)
298
299 ## ? PLAN: checkpoint the heavyweight WASB pass (resume across restarts) ? IMPLEMENT
NEXT SESSION
300 Problem: the GPU detector pass (21k forward passes) writes the trajectory ONLY at the
end, so a
301 reboot mid-pass loses all of it (happened 2026-06-06).
302 Key insight: **decouple the expensive GPU detector pass (checkpoint incrementally) from
the cheap,
303 stateful CPU tracker pass (just re-run from the cache).** The online tracker can't
resume
304 mid-stream, but if per-frame DETECTOR outputs are cached, the tracker re-runs over them
in seconds.
305 Plan (prioritised):
306 1. **Stable cache dir keyed by video MD5** (not %TEMP%): `output/wasb_cache/<md5>/` with
`frames/`,
307 `det_raw.jsonl` (incremental detector outputs), `trajectory.csv`, `manifest.json`
308 {video_md5, frames_total, frames_done, cfg}. (Ties to the indexer MD5-keying gotcha.)
309 2. **Idempotent extraction:** skip if frame count already == expected.
310 3. **Incremental detector cache (the big win):** in wasb_infer's batch loop, append each
batch's
311 per-frame raw detections to `det_raw.jsonl` and FLUSH every ~1000 frames; on startup
read
312 manifest/det_raw ? skip already-done frames ? resume. Loss bounded to <1000 frames
(~secs of GPU).
313 4. **Tracker re-run:** once all frames are detected (fresh or resumed), run the tracker
over the full
314 `det_raw.jsonl` in order ? `trajectory.csv`. Cheap, deterministic, re-runnable.
315 5. **Atomic writes** (write .tmp then rename) so a crash mid-write can't corrupt the
cache.
316 6. (Optional) expose as one resumable CLI stage.
317 Effort: moderate ? mostly a wasb_infer refactor (incremental det cache + resume); the
tracker
318 decoupling is the core design. Calibration grid stays cheap (operates on the cached
trajectory) ?
319 no checkpointing needed there.
320
321 ## ? STEP 4 ? FIRST CALIBRATION DONE (2026-06-06) ? RESULT
322 Full WASB trajectory of the long badminton video: 21,284 frames, 37% visible-shuttle
(sane).
323 `calibrate_wasb` on 25 golden rallies (IoU 0.5):
324 - **baseline** thresholds (0.02,2.0,2.0): precision 0.577, recall 0.600, **F1 0.588**,
meanIoU 0.765.
325 - **tuned** (0.05,1.0,1.0): **F1 0.746** ? precision +12.2%, **F1 +26.8%** (OPTIMISTIC ?
grid fit on full GT).

```

```

326 - **HONEST (5-fold cross-val): held-out recall 0.88 ± 0.16, generalization_gap +0.000**
(no overfit).
327 - Caveats: +X% is optimistic upper bound; honest within-video metric is RECALL;
precision/F1
328 generalization needs a 2nd labelled video ? `leave_one_video_out`. std ±0.16 = small
sample.
329 - Driver `backend/eval/calibrate_wasb.py` was LOST from master in #21's branch-delete ?
RECOVERED
330 in PR #24 (merged). Trajectory cached at `~/clips/sample_long_traj.csv` (+ `C:\?\Temp\
331 sample_long_traj.csv`); frames at `C:\?\Temp\sample_long_frames` (4.1GB, deletable).
332 - NEXT: export tuned segments for user manual-verify; 2nd labelled video for honest
precision;
333 implement WASB-pass checkpointing (plan below); optional multi-agent tuning Workflow.
334
335 ## (historical) STEP 4 IN PROGRESS (2026-06-06)
336 - PR **#21** open (feat/wasb-substrate: substrate + calibration.py + `calibrate_wasb.py`
driver +
337 HOW_RALLY_DETECTION_WORKS doc). Suite 171 green.
338 - **Calibration driver DONE + tested:** `backend/eval/calibrate_wasb.py` (grid over
339 velocity_thresh/merge_gap/min_window_duration ? improvement_report + cross_validate).
Run:
340 `python -m backend.eval.calibrate_wasb --trajectory ~/clips/sample_long_traj.csv (copy
to Win)
341 --labels "<Annotation Setup>/golden_labels_badminton.csv" --fps 59.94 --frame-width
1920`.
342 - **GOTCHA:** WSL CANNOT see Google Drive `/mnt/g` (first pass b04odl4jl failed: "cannot
stat
343 /mnt/g/..."). Fix: copy the video G:C: on the Windows side first (`cp "/g/My
Drive/.../Sample
344 Long Video - Baadminton.MP4" /c/Users/avidu/AppData/Local/Temp/sample_long.mp4`, ~9s,
cached),
345 then WSL reads it via `/mnt/c`.
346 - **PERF GOTCHA:** wasb_infer's cv2 `imwrite` PNG extraction is pathologically slow
(~5fps ? ~1hr
347 for 21k frames, GPU idle the whole time). FIX: extract with **ffmpeg** (Windows, reads
the C:
348 copy): `ffmpeg -i C:\...\sample_long.mp4 -q:v 2 C:\...\sample_long_frames\%08d.jpg`
(21284 frames
349 in **3.5 min**, 4.1GB JPG). Then `wasb_infer --frames_dir` (now defaults
**batch_size=8,
350 num_workers=4** ? GPU ~80-90%). ffmpeg NOT in WSL; use Windows ffmpeg. wasb_infer
globs *.jpg too.
351 - **RESTART RECOVERY (2026-06-06):** laptop rebooted mid-inference. The 21,284 extracted
frames
352 (`C:\Users\avidu\AppData\Local\Temp\sample_long_frames`) + staged video survived (Temp
persisted);
353 trajectory CSV was NOT written (writes only at end). Resume = just RE-RUN the
inference command
354 below on the existing frames (NO re-extraction). Relunched as task brmhzav6x. The
355 `wasb_infer.py` batch/workers change is uncommitted on master working tree (commit
after verify).
356 - **Full-video WASB inference RUNNING** (task brmhzav6x; orig b4pqhqkte died on reboot):
`wasb_infer.py --frames_dir
357 /mnt/c/Users/avidu/AppData/Local/Temp/sample_long_frames --weights
358 ../pretrained_weights/wasb_badminton_best.pth.tar --out ~/clips/sample_long_traj.csv
359 --batch-size 8 --num-workers 4`. GPU confirmed ~80-90%, 6.4/8GB. Writes CSV at the
end. When done:
360 copy CSV to Windows (or read /mnt) ? run the driver above ? the "+X%" + cross-val.
Then clean
361 up `~/clips/sample_long_frames` (~30-40GB). Calibrate ONLY on the labelled Sample Long
Video
362 (only one with golden labels); a 2nd labelled video later ? leave_one_video_out.
363
364 ## NEXT-SESSION RUNBOOK ? step 4 (the actual calibration run)
365 Substrate (wasb_infer/runner/hybrid) + eval harness (calibration.py) are DONE on

```

```

366     `feat/wasb-substrate` (pushed). To summit:
367 1. **Full-video WASB trajectory (the slow long pole ? do first, cache it).** Untested at
full
368     scale; expect a debug lap. Two paths:
369     (a) via runner: `WasbRunner(WasbConfig()).run_predict(<video>, "output/wasb")` ?
stages 2.8GB
370     into WSL (off G: = possibly slow/flaky) + extracts frames + runs. OR
371     (b) safer: extract frames on Windows (ffmpeg from G:, proven) ? run in WSL:
372     `conda activate wasb && cd ~/models/WASB-SBDT/src && python wasb_infer.py
--frames_dir <dir>
373     --weights ../pretrained_weights/wasb_badminton_best.pth.tar --out
~/clips/full_traj.csv`.
374     Watch: disk for ~21k PNGs, GPU mem, /mnt read speed. Validate visible-shuttle % is
high during
375     known rallies, ~0 during dead time.
376 2. **GT** from `Annotation Setup/golden_labels_badminton.csv` ? rally [start,end] in
seconds
377     (use the verified rows; skip the 1 missed row whose end is still TBD = rally 7.5).
378 3. **Write the calibration driver** (not yet written): `predict_fn(cfg)` =
parse_trajectory_csv(full_traj)
379     ? `TrackNetRunner.trajectory_to_action_windows(points, fps=59.94, frame_width=1920,
**cfg)` ?
380     [start,end]; cfg = {velocity_thresh, merge_gap, min_window_duration}. Grid over
those.
381 4. **Run** `backend/eval/calibration.py`: `improvement_report(predict_fn, baseline_cfg,
best_cfg, GT)`
382     for the "+X%"; `cross_validate` (within-video recall + gap) for the overfit check.
(Only 1 video
383     now ? leave_one_video_out waits for more videos = the real generalization metric.)
384 5. Optional: the **multi-agent tuning Workflow** (token-heavy) instead of a plain grid
in step 3.
385
386     ## Eval/overfit harness BUILT + license soul-check (2026-06-06)
387     - `backend/eval/calibration.py` (on `feat/wasb-substrate`, 9 tests; segmenter-agnostic
predict_fn):
388     `improvement_report` = baseline-vs-tuned %? precision/F1 (the "+X%"); `cross_validate`
=
389     within-video k-fold, default metric **recall** (global windows vs a GT *subset* can't
fairly
390     score precision), reports `generalization_gap` (train?test = overfit alarm);
`leave_one_video_out`
391     = the honest metric as videos arrive (pick config on others, score held-out). This is
the
392     overfitting guard for "more videos coming."
393     - **LICENSE:** CODE clean ? WASB-SBDT MIT (verified), torchvision/supervision
permissive, AGPL
394     Ultralytics gone (PR #8), MonoTrack never referenced (default = wasb_badminton_best).
?? OPEN:
395     WASB *weights* trained on academic NYCU/TrackNetV2 data ? confirm terms or retrain
before
396     COMMERCIAL deploy (fine for R&D). See [[tracknet-wasb-wsl2-decision]].
397
398     ## Step-3 substrate finding (2026-06-06)
399     - WASB IS the plan and is ~ready: `wasb` WSL env + WASB-SBDT repo + pretrained badminton
weights
400     all present (verified); healthcheck OK (TF2.17/GPU). The wired `tracknet_runner`
targets
401     TrackNetV4 = NOT runnable (no weights, repo not cloned). To use WASB: build inference
wrapper +
402     runner adapter + `wasb_hybrid` segmenter, then calibrate
`velocity_thresh/merge_gap/min_window_duration`
403     vs golden labels (eval held-out). See [[tracknet-wasb-wsl2-decision]] STATUS
2026-06-06.
404
405     ## Merged: #6, #7, #19, #8, #20, **#9 (validate-delivery)**

```

```

406 - collector master has: adapter, forced/unforced vocab, last_shot_outcome (two-field).
407 - indexer master has: docs mirror, rally slicer (`backend/tools/rally_slicer.py`).
408
409 ## Open PR (pushed, awaiting merge)
410 - **collector #9** `feat/validate-delivery` ? per-video QA gate. `curate validate-
delivery`
411 (CVAT or CSV) ? `_report.md`/`.json` + `_review.csv` (auto manual-verify
sheet);
412 exit!=0 on blocker. New `possible_missed_rally` gap check.
`docs/INGESTION_PIPELINE.md` FAQ.
413 104 tests; verified on real delivery (BLOCKED on 1?2 overlap, flags 2/7/16).
414
415 ## Pilot status ? DONE verifying
416 - User verified **all** rallies: only the previously-flagged rows are issues;
**everything else
417 is clean trainable data**. The [start,end] intervals are settled ? **segmentation GT
is ready**
418 (ending_reason/outcome specifics for flagged rows are metadata, not needed for
temporal seg).
419 - Golden master: `C:\Users\avidu\Projects\Annotation Setup\golden_labels_badminton.csv`
420 (24 + 2 missed = rallies; 7.5 @1:46.031 confirmed via screenshot; 16.5 = 3:34?3:43).
421
422 ## Decisions locked
423 - **Two-field ending**: `ending_reason` (why) + `last_shot_outcome` ? {in,net,out,n_a}
(where).
424 - **Accept Roora's CVAT export** ? adapter ingests it; do NOT ask them to switch to CSV.
425 - **Absorb timestamps** ? adapter recomputes `seconds = frame/fps`; mm:ss also parsed
everywhere
426 (indexer parse_timestamp / collector parse_time_string / compiler / slicer). Minor
slack
427 absorbed by VideoCompiler padding (0.5/1.0s). Don't ask Roora to fix time computation.
428 - Roora report (`Annotation Setup\Roora Pilot Review ? Badminton (2026-06-06).md`)
revised to
429 only ask for what code can't fix: **completeness (no missed rallies), one-box-per-
rally
430 boundaries, add `last_shot_outcome` dropdown, pickleball/TT scope.**
431
432 ## Next steps (sequence 1?2?3, user approved)
433 > User will get **5?6 30-min+ videos** annotated soon ? the **validation phase must be
ready**
434 > to auto-check each delivery against our conventions BEFORE that lands.
435 1. ? **last_shot_outcome vertical** ? MERGED (#8).
436 2. ? **validate-delivery framework** ? DONE (collector PR #9, awaiting merge). Label-
only gap
437 heuristic for missed rallies (a motion-based 2nd opinion over raw video is a future
enhancement).
438 3. (claude) **NEXT: per-video calibration loop** ? slicer merged (#20); tune segmenter
thresholds
439 on K labeled rallies, measure IoU on the HELD-OUT rest (no leakage). Then learned
fine-tuning.
440 4. (claude) Build segmentation GT from golden labels ? convert-cvat/import-local/export
(the
441 actual "train on my own video" step; needs the corrected golden labels, overlap
already resolved).
442
443 ## Open design threads
444 - Per-video few-shot adaptation: calibration first (static camera = good case), honest
eval
445 (held-out rallies). cf. train/eval guardrail in [[collector-indexer-bridge]].
446

```

4. user (2026-06-06T17:28:15.074Z)

<system-reminder>This memory is 7 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against

```

current code before asserting as fact.</system-reminder>
1      ---
2      name: collector-indexer-bridge
3      description: "Plan to feed sports-data-collector's rally annotations into the indexer to
train/eval a generic temporal rally segmenter"
4      metadata:
5          node_type: memory
6          type: project
7          originSessionId: 7572cbc9-3822-461e-8779-c8cec498cde9
8      ---
9
10     Goal: use `sports-data-collector`'s commercially-licensed rally annotations to **train
and evaluate a generic temporal rally segmenter** in `badminton-highlight-indexer`, reusable
across sports (badminton/tennis/cricket share a `(video_id, ordinal, start_time, end_time)`
shape).
11
12     Key reframing established 2026-05-26: the indexer's `yolo_hybrid` segmenter uses **a
stock person detector (person class only) + a velocity heuristic** ? there is no trained
badminton model. (As of 2026-05-30 that detector is **torchvision Faster R-CNN + supervision
ByteTrack**, swapped off AGPL Ultralytics YOLOv8 ? PR #8/ADR-005; the velocity-heuristic logic
is unchanged.) The collector's data is **temporal-only** (rally `[start,end]`, stroke counts,
scores; NO per-frame shuttle `(x,y)`). So this data trains/tunes the **temporal segmentation**
stage, NOT a shuttle object detector (that needs coordinate labels ? see [[tracknet-wasb-
wsl2-decision]] and [[candidate-segments-finetuning]]).
13
14     Golden-set ground truth for eval (beyond the collector's own annotations) is now planned
via **third-party annotation vendors** on licensed/owned video ? design + vendor shortlist in
[[golden-set-vendoring]].
15
16     Phased plan (user picked: train a temporal segmenter; exporter lives as a collector CLI
subcommand):
17     - **Phase 1 (DONE):** `python -m src.cli.curate export` in sports-data-collector. Emits
`<out>/<sport>/<video_id>.csv` (indexer `start,end` format) + `manifest.json`. Filters curated +
`is_commercial` by default. Code: `SportsDatabase.get_segment_intervals()` + `SEGMENT_TABLES`
map in `src/core/db.py`; `export_eval_set()` in `src/cli/curate.py`.
18     - **Phase 2 (code DONE; bulk re-fetch BLOCKED on cookies):** on-disk videos were
**256x144** because the crawler hard-coded `yt-dlp -f worstvideo`. Fixed:
`src/core/downloader.py` always fetches best resolution (`bv*+ba/b -S res,...`, merged mp4,
ffprobes actual w/h/fps, pins `youtube:player_client=web_safari`, enables EJS solver `--remote-
components ejs:github`, auto-adds `~/deno/bin` to PATH). New `curate fetch` re-downloads
existing DB entries at best res, updates `local_path`/`resolution`/`fps` **in place** (rallies +
curation preserved ? NO clean slate). Guard: below-min-height downloads are discarded, not
committed. deno 2.8.0 installed. Both repos now on GitHub under `avidullu` (badminton-highlight-
indexer; sports-data-collector created PRIVATE 2026-05-26); both `master` in sync with origin.
19     - **RESOLVED + COMPLETE (2026-05-26):** root cause of persistent 144p was the
`web/web_safari` clients being forced into SABR streaming (strips download URLs); fix =
`player_client=default` + authenticated Firefox Premium cookies (`--cookies-from-browser
firefox`; Firefox reads cleanly on Windows, Chrome blocked by DPAPI app-bound encryption
#10927). Bulk batch initially failed all-at-once from YouTube rate-limiting ? added
`--sleep`/`--retries`/`--retry-backoff` to fetch. Final result: **all 22 fetchable badminton
matches now true 1920x1080** (verified on disk, rallies intact), DB committed. Unfetchable:
shuttleset_12 (no URL in ShuttleSet catalog), my_custom_local_match + staging_match_non_comm
(mock/non-commercial). ~26GB in data/videos (gitignored). ShuttleSet corpus tops out at 1080p
(no 4K). **Phase 2 done.**
20     - **Phase 3 (IN PROGRESS):** eval/tune loop. Built `run_phase3_eval.py` +
`backend/eval/batch.py` in indexer (walks manifest, processes each video, scores vs GT, per-
video + corpus aggregate; pure helpers tested). Loop validated on 1 video with `motion`.
Committed on indexer branch `feat/phase3-eval-and-offline-roadmap`. Indexer `output/` is
gitignored. NOTE: indexer validators gate on min_fps 29.9 but ShuttleSet videos are 25fps ? the
driver calls the segmenter directly, bypassing the validation gate.
21     - **Phase 3 EVIDENCE (2026-05-26):** ran motion (shuttleset_1) + yolo_hybrid
(shuttleset_8). Findings: (1) Gemini key is FREE TIER (20 req/day) ? 429 quota ? API path dead,
pivot vindicated. (2) Processing is SLOW (~50min per 1080p video). (3) **YOLO candidate recall
0.05@IoU0.5** (0.72@0.01; 72% rallies have some overlap but median coverage ~20%); **merging
candidates does NOT help**. Root cause: broadcast camera TRACKS players ? player pixel-velocity

```

is a poor rally proxy (designed for static GoPro). `motion` also useless (0 TP). ? **Offline** substrate decided: WASB shuttle-motion (ADR-001), NOT YOLO. **Caveat:** 1 video, confirm on 2-3 more via free candidates-only YOLO pass. Already-built training-data + classifier layers (`backend/eval/training.py`, `classifier.py`, committed on branch `feat/phase3-eval-and-offline-roadmap`) are substrate-agnostic ? will consume WASB windows/features.

22 - **Phase 4 (NEXT ? direction set 2026-05-26):** build an OFFLINE, API-free learned segmenter (e.g. `yolo\_learned`) ? replace the Gemini refinement with a lightweight classifier trained on rally annotations + already-stored `candidate\_segments.stats`. User explicitly wants NO external API. Substrate (YOLO player-motion features vs WASB shuttle-motion) decided by the **candidate-stage recall** from the running yolo_hybrid validation: high recall ? YOLO features; low ? WASB. Full paid 22-video Gemini eval NOT run (only 1 reference video). Decisions documented: indexer `docs/ROADMAP.md` + `docs/DECISIONS.md` ADR-002/003/004; collector `docs/PIPELINE\_ROLE.md`.

23
24 **Critical integration gotcha:** indexer keys videos by **file MD5**, collector by human `video\_id`. Re-encoding/downloading changes bytes?MD5, so acquisition must precede processing and the manifest tracks which exact file was used.

25

5. assistant (2026-06-06T17:28:18.993Z)

I have enough context. The project is a self-hosted, local-first badminton rally segmentation pipeline with WASB shuttle trajectory (MIT), a permissive person detector, and an audio track available, currently using Gemini-teacher ? local-distillation with F1 ceilings around 0.44. Now I'll decompose the research question into complementary search angles.

6. user (2026-06-06T17:28:36.999Z)

Structured output provided successfully

7. assistant (2026-06-06T17:28:43.457Z)

I've decomposed the research question into 5 complementary search angles, grounded in the project's actual constraints (WASB MIT substrate, RTX 2070 Super, local-first, commercial licensing, current Gemini-distillation at F1 ? 0.44).

The angles are:

1. **broad/primary** ? the overall method-family landscape and accuracy baselines for rally/point segmentation.
2. **practitioner/local-levers** ? the two fully-local approaches the user most wants assessed: audio hit detection and pose/court-geometry serve-stance state machines.
3. **classical/lightweight** ? change-point/HMM on the existing shuttle trajectory plus scoreboard OCR for point boundaries (cheap, CPU-friendly).
4. **datasets/licensing** ? ShuttleSet and TrackNet/TrackNetV2 dataset terms, the make-or-break commercial gate.
5. **contrarian/distillation-vs-weak** ? adversarially testing the current VLM-teacher ? student plan against weak/self-supervised methods given the few-labels overfitting they're already hitting.

A 6th applied/recent angle is included to catch newer permissive on-device models across sibling racket sports.

The structured output has been returned successfully.